COMP3311 Week 4 Thursday Lecture

- Week 04 Thursday
- Assignment 1 Marking
- Assignment 1 Hints
- Exercise: Monitor progress of factorial loop
- Iteration in PLpgSQL
- Exercise: Implement this function more efficiently
- Exercise: Poor use of loop
- Returning Multiple Results
- Exercise: Withdrawal Function returning two values
- Aggregates
- Exercise: User-defined Aggregates
- Global Constraints
- Triggers
- Triggers in PostgreSQL
- Exercise: Trigger Example (I)
- Exercise: Trigger Example (II)
- Programming with Databases
- Introduction Java/JDBC/Perl

◆ Week 04 Thursday

In today's lecture ...

- More PLpgSQL
- Aggregates, Constraints, Triggers

Things to do ...

- Census: TODAY is last day to withdraw without paying
- Quiz 3 due before Friday midnight (Assignment Hints)
- Assignment 1, due before midnight on Friday 21 March
  - 350 / 10 submissions so far ... haven't checked their correctness!

◆ Assignment 1 Marking

This how we will test your submission for marking:

```
psql TempDB1 -f create2dt TempDB1
psql TempDB1 -f sed1.dump
psql TempDB1 -f sed1.sql [input stdin]
psql TempDB1 -f unseen.dump
psql TempDB2 -f create2dt TempDB2
psql TempDB2 -f unseen.dump
psql TempDB2 -f sed1.sql [input stdin]
... run tests on TempDB1 ...
```

Make sure your code will work if loaded like this on **vxdb02**

Of course, not including **unseen.dump** and **TempDB2**

◆ Debugging PLpgSQL

PLpgSQL does not have I/O functions

How to do debugging (we intermediate states)?

Use PostgreSQL logging statement

```
raise notice 'Message' [, expr, .Expr, ... ]
```

Examples:

```
declare x integer; y float;
...
raise notice 'x = %', x;
raise notice 'x = %, y = %', x, y
raise notice 'x^2 = %', x*x;
```


◆ Exercise: Monitor progress of factorial loop

Modify the following function to show intermediate factorial values:

```
create or replace function
  f(acin numeric) returns numeric
as $$
declare
  i integer;
  fac numeric;
begin
  fac := 1;
  for i in 1 .. n loop
    fac := fac * i;
  end loop;
  return fac;
end;
$$ language plpgsql;
```


◆ Iteration in PLpgSQL

Don't do only code what also could do using SQL

E.g. use **where** clause to avoid tests within loops

```
for T in select * from Table
loop
  if T.x > 0 then
    processed T ...
  end if;
end loop;

-- should be implemented as

for T in select * from Table where x > 0
loop
  processed T ...
end loop;
```


◆ Iteration in PLpgSQL (cont)

E.g. use **join** operation to avoid nested loops

```
for T in select * from T1
loop
  for S in select * from T2
  loop
    if T.a = S.y then processed T+S ...
  end loop;
end loop;

-- should be implemented as

for Res in
  select * from T join S on A.a=S.y
loop
  processed Res ...
end loop;
```


◆ Exercise: Implement this function more efficiently

Example: function returning **setof Beers**

```
create or replace function
  bn(_name text) returns setof Beers
as $$
declare
  b Beers;
begin
  for b in select * from Beers
  loop
    if b.name = _name then
      return next b;
    end if
  end loop;
end;
$$ language plpgsql;
```


◆ Exercise: Poor use of loop

How many iterations of the loop in each of the following?

```
  for rec in
    select * from R
    loop
      if (rec.a = 5) then
        ...
      end if;
  end loop;

OR

  for rec in
    select * from R where a = 5
    loop
      ...
  end loop;
```

Assume: |R| = 10000 tuples, 10 tuples with **a=5**

◆ Returning Multiple Results

Already known: returning a set of values **returns setof** *Type*

What about a function that returns two different types of value?

E.g. withdrawal function returns new balance and a message

```
-- initial balance in account 777-432 is $2000

select withdrawal('777-432', 500, ...)
-- should "bounce" or not ...

select withdrawal('777-432', 10000, ...)
-- should "bounce" or not ...
```

Could use **OUT** parameters (parameter assigned a value within function)

◆ Returning Multiple Results (cont)

Simple example of using **OUT** parameters:

```
create or replace function
  sum_n_product(n int, p int, OUT sum int, OUT prod int)
as $$
begin
  sum := n + p;
  prod := n * p;
end;
$$ language plpgsql;

dbms# select sum_n_product(3,4);
 sum | prod
-----+------
   7 |   12
(1 row)
```

Effectively creates an anonymous tuple type **(sum,prod)**

◆ Exercise: Withdrawal Function returning two values

Show how the

```
withdrawal(acct text, Amount Money) ...
```

function could be implemented

- using **OUT** parameters
- using an explicit tuple type

Reminder: returns new balance and a message

◆ Aggregates

Aggregates reduce a collection of values to a single value

Example:

```
select avg(mark) from Enrolments where course='COMP3311'
```

How to achieve this? ... Maintain state, update value by value

```
$state = initial state
for each tuple T in Query {
  -- update state
  $state = updateState($state, T)
}
return makeFinal($state)
```


◆ Aggregates (cont)

New aggregates are defined using **CREATE AGGREGATE** statement:

```
CREATE AGGREGATE AggName(BaseType) (
    stype     = StateType,
    initcond  = InitialValue,
    sfunc     = UpdateStateFunction,
    finalfunc = MakeFinalFunction
);
```

- **initcond** (type StateType) is optional; defaults to NULL
- **finalfunc** is optional; defaults to identity function

◆ Exercise: User-defined Aggregates

1. make your own **count** aggregate

```
  select mycount(a) from R - 3
  select mycount(b) from R - 2
```

2. make a **product** aggregate

```
  select product(a) from R - 6
```

3. simple string concatenation (comma-separated)

```
  select cat(b) from R - 'this,is,fun'
```

Assuming

```
create table R (a integer, b text);
insert into R values (1,'this'); (2,'is'); (3,'fun');
```


◆ Global Constraints

Column and table constraints ensure validity of one table

Ref. integrity constraints ensure connections between tables are valid

Global constraints may involve conditions over many tables

Simple example (from banking domain):

```
-- accounts are held at branches
-- assets of branch is sum of balances of its accounts

for all Branches b:
  b.assets == (select sum(acct.balance)
               from   Accounts acct
               where  acct.branch = b.location)
```


◆ Global Constraints (cont)

SQL implementation of global constraints is **ASSERTION**

Example: #students in any UNSW course must be < 10000

```
create assertion ClassSizeConstraint check (
  not exists (
    select c.id
    from   courses c
           join Enrolments e on (c.id = e.course)
    group  by c.id
    having count(e.student) > 9999
  )
);
```

Must be checked after every change to either **Courses** or **Enrolments**

Too expensive, so DBMSs provide triggers to do targetted checking.

◆ Triggers

Triggers are

- procedures stored in the database
- invoked in response to database events (e.g. insert/update/delete)

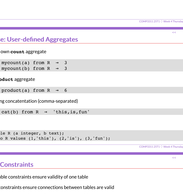
Examples of uses for triggers:

- maintaining summary data
- checking schema-level constraints (assertions) on update
- performing multi-table updates (to maintain assertions)

Triggers provide event-condition-action programming

◆ Triggers (cont)

Sequence of activities during database update:



Reminder: **BEFORE** and **UPDATE** triggers can modify value of new tuple

◆ Triggers in PostgreSQL

PostgreSQL triggers provide a mechanism for

- **INSERT**, **DELETE** or **UPDATE** events
- to automatically activate PLpgSQL functions

Syntax for PostgreSQL trigger definition:

```
CREATE TRIGGER TriggerName
[AFTER|BEFORE] Event1 [OR Event2 ...]
ON Table1Name
[ WHEN ( Condition ) ]
FOR EACH [ROW|STATEMENT]
EXECUTE PROCEDURE FunctionName(arga,...);
```

- **BEFORE** triggers must return **old** or **new**
- any exception during a trigger forces roll-back

◆ Exercise: Trigger Example (I)

Consider two tables

```
create table X (id integer, val text);
create table Y (id integer references X(id), value text);
```

Write a trigger to check that the foreign key value is valid

- when you insert a new a tuple
- when you update an existing a tuple

❖ Exercise: Trigger Example (ii)

Class enrolments:

```
Classes(id,name,room,day,start,end,quota,nstu)
ClassEnrolments(student,class)
```

Define triggers to maintain **nstu** for

```
insert into ClassEnrolments values (5012345, 6732);

insert into ClassEnrolments values (9999999, 99999);

delete from ClassEnrolments
where  student = 5012345 and class = 6732;
```


❖ Programming with Databases

So far, we have seen ...

- accessing data via SQL queries
- packaging SQL queries as views/functions
- building functions to return tables
- implementing assertions via triggers

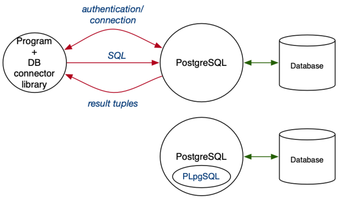
All of the above programming

- is very close to the data
- takes place inside the DBMS

For applications need a PL interacting with the DBMS.

❖ Programming with Databases (cont)

Programming Language / DBMS archtecture:



❖ Programming with Databases (cont)

Consider this (imaginary) PL/DBMS access method:

```
--  establish connection to DBMS
db = dbAccess("DB");
query = "select a,b from R,S where ... ";
--  invoke query and get handle to result set
results = dbQuery(db, query);
--  for each tuple in result set
while (tuple = dbNext(results)) {
    --  process next tuple
    process(tuple['a'], tuple['b']);
}
```

Estimated costs: **dbAccess** = 500ms, **dbQuery** = 100ms, **dbNext** < 1ms

❖ Iteration over Data in PLs

Example: find mature-age students (e.g. 10000 students, 500 over 40)

```
# assume we have a DB handle
query = "select * from Student";
results = dbQuery(db, query);
while (tuple = dbNext(results)) {
    if (tuple['age'] > 40) {
        --  process mature-age student
    }
}
```

We transfer 10000 tuples from DB, 9500 are irrelevant

Cost = 1 query + 10000 tuple fetches = 1*100 + 10000*1 = 10100 ms

❖ Iteration over Data in PLs (cont)

Should be implemented as:

```
# assume we have a DB handle
query = "select * from Student where age >= 40";
results = dbQuery(db, query);
while (tuple = dbNext(results)) {
    --  process mature-age student
}
```

Transfers only the 500 tuples that are needed.

Cost = 1 query + 500 tuple fetches = 1*100 + 500*1 = 600 ms

❖ Iteration over Data in PLs (cont)

Example: find info about all marks for all students

```
# assume we have a DB handle
query1 = "select id,name from Student order by id";
res1 = dbQuery(db, query1);
while (tuple1 = dbNext(res1)) {
    query2 = "select course,mark from Marks"
             + " where student = " + tuple1['id'];
    res2 = dbQuery(db,query2);
    while (tuple2 = dbNext(res2)) {
        --  process student/course/mark info
    }
}
```

E.g. 10000 students, each with 8 marks, ⇒ run 10001 queries

Cost = 10001 queries + 80000 tuple fetches = 100*10001 + 1*80000 = 1080100 ms

❖ Iteration over Data in PLs (cont)

Should be implemented as:

```
# assume we have a DB handle
query = "select id,name,course,mark"
        + " from Student s join Marks m "
        + " on (s.id=m.student)"
        + " order by s.id"
results = dbQuery(db, query);
while (tuple = dbNext(results)) {
    --  process student/course/mark info
}
```

We invoke 1 query, and transfer same number of tuples.

Cost = 1 query + 80000 tuple fetches = 1*100 + 80000*1 = 80100 ms